

Subspace-Boosted Model Merging

好的，本篇论文的详细解读如下。

论文概述

[cite_start]这篇名为《Subspace-Boosted Model Merging》的论文探讨了在模型合并领域中的一个关键问题。模型合并旨在将多个经过微调的“专家”模型的功能整合到一个单一模型中，从而节省存储和计算成本 [cite: 1, 10, 11][cite_start]。然而，研究人员发现，随着合并的专家模型数量增加，整体性能的提升会遇到瓶颈，甚至出现性能下降的现象 [cite: 2]。

论文的核心贡献可以概括为以下几点：

- [cite_start]**发现并分析了“秩崩溃” (Rank Collapse) 问题**：论文从任务算术 (task arithmetic) 的角度分析，揭示了在合并越来越多专家模型的过程中，任务向量空间的“秩”会发生崩溃，这是导致性能下降的根本原因 [cite: 3, 4]。
- [cite_start]**提出了 Subspace Boosting 方法**：为了解决秩崩溃问题，论文引入了一种名为“子空间提升” (Subspace Boosting) 的新方法。该方法通过奇异值分解 (Singular Value Decomposition, SVD) 来操作任务向量，并提升特定维度，从而保持任务向量的秩，显著提升了合并多达20个专家模型时的性能，在视觉基准测试中提升超过10% [cite: 5, 6]。
- [cite_start]**引入了基于 HO-GSVD 的可解释性方法**：为了进一步分析和理解模型合并过程，论文提出使用高阶广义奇异值分解 (Higher-Order Generalized Singular Value Decomposition, HO-GSVD)。这种方法不仅可以量化不同任务之间的相似性，还能分解出共享和独特的任务子空间，为选择哪些专家模型进行合并提供了新的可解释视角 [cite: 7, 33, 39]。

核心问题：模型合并中的秩崩溃

1. 任务算术与任务向量

[cite_start]首先，我们需要理解“任务算术”的背景 [cite: 43]。

- 假设我们有一个预训练的基础模型 M_{base} ，其权重参数为 θ_{base} 。
- 我们有一组 (n 个) 专家模型 $\{M_1, M_2, \dots, M_n\}$ ，它们都是在 M_{base} 的基础上针对不同任务微调得到的，其对应的权重参数为 $\{\theta_1, \theta_2, \dots, \theta_n\}$ 。
- [cite_start]**任务向量 (Task Vector)** 被定义为专家模型权重与基础模型权重之差 [cite: 61]：
$$\Delta_i = \theta_i - \theta_{base}$$

这里的 Δ_i 就代表了模型为了掌握第 i 个任务而学习到的“知识”或“方向”。
- [cite_start]最简单的模型合并方法——线性插值（或任务算术），其合并后模型的参数 θ_m 可以表示为 [cite: 62]：
$$\theta_m = \theta_{base} + \alpha \sum_{i=1}^n \Delta_i$$

[cite_start]其中， α 是一个标量合并系数。我们将所有任务向量的总和定义为总任务向量
$$\Delta_m = \sum \Delta_i$$
 [cite: 67]。

2. 秩崩溃 (Rank Collapse) 现象

[cite_start]论文的关键洞见在于，当合并的专家模型越来越多时（即 n 增大），总任务向量 Δ_m 的矩阵会发生秩崩溃 [cite: 21]。

- [cite_start]**什么是秩崩溃？** 秩崩溃是指矩阵中的大部分信息被集中在最大的少数几个奇异值和对应的奇异向量中 [cite: 22][cite_start]。这会导致矩阵的“有效秩”远低于其理论最大秩，意味着权重矩阵实际上只在一个非常有限的子空间中运作 [cite: 25]。

- **如何衡量？** 论文使用了两个指标来衡量秩崩溃：

1. [cite_start]**稳定秩 (Stable Rank)**：这个指标衡量矩阵的“有效秩”，计算方式为 [cite: 76]：

$$\mathcal{M}_{stable} = \frac{\sum_i \sigma_i^2}{\max_i \sigma_i^2}$$

其中 σ_i 是矩阵的奇异值。稳定秩忽略了那些非常小的奇异值的影响。

2. [cite_start]**累积能量秩 (Cumulative Energy Rank)**：该指标计算需要多少个最大的奇异值才能覆盖矩阵总“能量”的95% [cite: 73, 74][cite_start]。能量 $\mathcal{E}$ 的定义为所有奇异值的平方和 $\mathcal{E} = \sum_i \sigma_i^2$ [cite: 74]。

- [cite_start]**实验观察**：如论文图1所示，单个微调模型的任务向量（如Cars, Food, Flower）具有相对较高的稳定秩。但当使用不同的合并方法（如TA, TIES, CSM）将14个任务向量合并后，得到的总任务向量的稳定秩显著下降 [cite: 63, 64, 77][cite_start]。这证明了秩崩溃现象的存在，并表明合并后的模型被迫在越来越小的子空间中编码越来越多的任务信息，从而限制了其泛化能力 [cite: 80]。

方法一：Subspace Boosting

[cite_start]为了解决秩崩溃问题，论文提出了 Subspace Boosting，该方法直接作用于合并后的总任务向量 Δ_m ，因此可以和任何依赖任务向量的合并方法兼容 [cite: 82, 83]。

1. 核心思想

其核心思想是通过奇异值分解 (SVD) 来调整任务向量的奇异值谱，从而“提升”那些信息量较少但同样重要的维度，以维持矩阵的有效秩。

2. SVD 背景

[cite_start]对于一个矩阵 $A \in \mathbb{R}^{m \times n}$ ，其SVD分解为 [cite: 72]：

$$A = U \Sigma V^T$$

- $U \in \mathbb{R}^{m \times m}$ 是左奇异向量矩阵。
- $\Sigma \in \mathbb{R}^{m \times n}$ 是一个对角矩阵，对角线上的元素是奇异值 $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ 。
- $V \in \mathbb{R}^{n \times n}$ 是右奇异向量矩阵。

3. 算法流程 (Algorithm 1)

[cite_start]Subspace Boosting 的具体步骤如下 [cite: 96]：

1. [cite_start]**输入**：合并后的总任务向量 Δ_m (按层划分) 和一个“提升阈值”超参数 β [cite: 96]。
2. [cite_start]**SVD分解**：对任务向量中的每一个矩阵组件（例如Transformer中的线形层和注意力层矩阵） Δ_{c_i} 进行SVD分解，得到 U, S, V^T [cite: 84, 98]。
3. [cite_start]**计算阈值索引**：计算奇异值的累积和，并找到最后一个使得累积和占比小于或等于阈值 β （例如，论文中设为3%）的奇异值的索引 `clampIndex` [cite: 85, 86, 100]。这意味着排在该索引之前的奇异值包含了绝大部分信息。
4. [cite_start]**提升奇异值**：将从 `clampIndex` 开始的所有后续奇异值，都设置为 `clampIndex` 处奇异值的大小 [cite: 86, 100]。这一步是“提升”的核心，它人为地增加了较小维度的重要性，防止信息过度集中。
5. [cite_start]**重构任务向量**：使用修改后的奇异值矩阵 S^* 与原始的 U 和 V^T 重新计算任务向量组件 [cite: 87, 101]：
$$\Delta_{boost_{i_c}} = U \cdot S^* \cdot V^T$$
6. [cite_start]**输出**：将所有提升后的组件重新组合，得到最终的 boosted 任务向量 Δ_{boost} [cite: 97]。

[cite_start]实验结果表明，该方法非常有效，能够将合并模型的性能大幅提升（超过10%），并且让简单的任务算术（TA）方法也达到了极具竞争力的水平 [cite: 6, 117]。

方法二：基于 HO-GSVD 的可解释性合并

[cite_start]标准的SVD分解为每个矩阵生成各自的奇异向量，这使得在不同模型之间直接比较子空间变得困难 [cite: 91, 92]。为了实现更强的可解释性，论文引入了高阶广义奇异值分解 (HO-GSVD)。

1. HO-GSVD 核心优势

[cite_start]给定一组矩阵 $\{A_1, \dots, A_N\}$ ，HO-GSVD可以将每个矩阵 A_i 分解为 [cite: 104]:

$$A_i = U_i \Sigma_i V^T$$

[cite_start]关键在于，这里的右奇异向量矩阵 V 对所有输入矩阵是共享的(shared) [cite: 104, 107]

[cite_start]。这提供了一个共同的参考框架，使得比较不同任务向量在同一个子空间上的重要性成为可能 [cite: 109, 110]。

2. 对比与对齐矩阵 (Alignment Matrix)

[cite_start]有了共享子空间 V 后，可以通过比较不同任务向量对应的奇异值 $\sigma_{i,k}$ 和 $\sigma_{j,k}$ 来分析任务间的关系 [cite: 111]。

- 如果比率 $\sigma_{i,k}/\sigma_{j,k} \gg 1$ ，说明子空间维度 V_k 对任务 i 比对任务 j 重要得多。
- 如果比率 ≈ 1 ，说明该维度对两个任务同等重要，是它们的共同子空间。
- 如果比率 ≈ 0 ，说明该维度对任务 i 不重要，但对任务 j 很重要，是任务 j 的独特子空间。

[cite_start]基于此，论文定义了对齐矩阵 (Alignment Matrix) A ，其元素 $\langle A \rangle_{ij}$ 表示模型 i 和模型 j 之间的平均对齐度 [cite: 122][cite_start]。计算公式为 [cite: 124]:

$$\langle A \rangle_{ij} = \frac{1}{L} \sum_{l=1}^L \left(\frac{1}{M_l} \sum_{p=1}^{M_l} \left| \log \left(\frac{\sigma_{i,p}^{(l)} + \epsilon}{\sigma_{j,p}^{(l)} + \epsilon} \right) \right| \right)$$

- L 是模型中需要聚合的组件总数（如层数）。
- M_l 是第 l 个组件的广义奇异值数量。
- $\sigma_{i,p}^{(l)}$ 是模型 i 在第 l 个组件上的第 p 个广义奇异值。
- ϵ 是为了数值稳定而加入的极小常数。

[cite_start]这个对齐分数越高，代表两个模型依赖的子空间差异越大，互相干扰就越小，因此“对齐得越好”，合并它们时效果可能更佳 [cite: 121, 200]。

如何得到V&为什么可以用奇异值大小来判断空间维度对于哪个任务向量重要

根据论文内容，我们来详细解答这两个问题。

1. 为什么对应的奇异值大小可以用来判断子空间维度对于哪个任务重要？

这个判断的逻辑建立在高阶广义奇异值分解 (HO-GSVD) 的核心特性之上。

在HO-GSVD分解中，每个任务向量矩阵 A_i (代表第 i 个任务) 被分解为：

$$A_i = U_i \Sigma_i V^T$$

这里的每个部分都有其特定含义：

- A_i 是第 i 个任务的任务向量矩阵，代表了该任务独有的“知识”。
- V^T 是一个共享的右奇异向量矩阵，它的每一行（即 V 的列向量 V_k ）可以被看作是一个基础的、所有任务共有的“方向”或“特征维度”，我们称之为子空间维度。
- Σ_i 是一个对角矩阵，其对角线上的元素 $\sigma_{i,k}$ 是第 i 个任务在第 k 个共享子空间维度上的奇异值。
- U_i 是第 i 个任务独有的左奇异向量矩阵。

核心思想：奇异值 $\sigma_{i,k}$ 在这里扮演着**权重或缩放因子**的角色。当我们要重构任务 A_i 时，实际上是将共享的子空间维度 V_k 按照各自的权重 $\sigma_{i,k}$ 进行线性组合。

因此：

- [cite_start]如果某个奇异值 $\sigma_{i,k}$ **很大**，意味着对应的共享子空间维度 V_k 在构成任务 A_i 时占有**很高的权重**。这表明这个子空间维度对于任务 i 来说是**非常重要的** [cite: 112]。
- [cite_start]如果某个奇异值 $\sigma_{i,k}$ **很小或接近于零**，意味着对应的共享维度 V_k 对构成任务 A_i 几乎没有贡献。这表明这个子空间维度对于任务 i 来说是**不重要的** [cite: 112]。

论文中通过比较不同任务在同一个共享维度上的奇异值比率来说明这一点：

- [cite_start]当 $\sigma_{i,k}/\sigma_{j,k} \approx 1$ 时，说明维度 V_k 对于任务 i 和任务 j 同等重要，这是一个**共同子空间** [cite: 113]。
- [cite_start]当 $\sigma_{i,k}/\sigma_{j,k} \gg 1$ 时，说明维度 V_k 对任务 i 的重要性远大于任务 j [cite: 112]。

2. 这个共享的右奇异向量矩阵V是怎么得到的？

共享矩阵 V 并不是直接从单个任务向量中分解出来的，而是通过一个更复杂的、综合了所有任务信息的计算过程得到的。

根据论文描述，获取 V 的步骤如下：

1. [cite_start]**求解一个特征系统 (eigensystem)**： V 是通过求解一个特定矩阵 S_π 的特征系统得到的，即对 S_π 进行特征分解 (eigendecomposition) [cite: 105]。 V 就是 S_π 的特征向量矩阵。
2. [cite_start]**计算核心矩阵 S_π** ： 矩阵 S_π 被定义为所有任务向量对的“商”的算术平均值 [cite: 105]。其计算公式为：
$$S_\pi = \frac{1}{N(N-1)} \sum_{i=1}^N \sum_{j=i+1}^N (D_{i,\pi} D_{j,\pi}^{-1} + D_{j,\pi} D_{i,\pi}^{-1})$$
其中 N 是任务的总数。
3. [cite_start]**计算中间矩阵 $D_{i,\pi}$** ： 在计算 S_π 之前，需要为每个任务向量 A_i 计算一个中间矩阵 $D_{i,\pi}$ [cite: 107]。其公式为：
$$D_{i,\pi} = A_i^T A_i + \pi A^T A, \quad \pi \geq 0$$
[cite_start]这里的 A 是将所有任务向量 A_i 堆叠起来的大矩阵， π 是一个正则化参数，用于处理秩亏矩阵的情况 [cite: 107, 108]。

总结一下获取V的流程：

1. 首先，根据所有任务向量矩阵 $\{A_1, \dots, A_N\}$ 计算出每个任务对应的中间矩阵 $D_{i,\pi}$ 。
2. 然后，计算所有任务对 (i, j) 之间的商矩阵 $D_{i,\pi} D_{j,\pi}^{-1}$ 。
3. 将所有这些商矩阵进行平均，得到最终的综合矩阵 S_π 。
4. [cite_start]最后，对矩阵 S_π 进行特征分解。分解得到的**特征向量矩阵**就是我们需要的那个包含了所有任务共同子空间信息的共享右奇异向量矩阵 V [cite: 105]。

3. 应用：最优专家选择

[cite_start]对齐矩阵的一个直接应用是**专家选择** [cite: 126][cite_start]。当需要从一个大的专家模型池中选择一部分进行合并时，可以通过最大化所选模型之间的平均对齐分数来做出选择，从而减少合并时的任务干扰。实验证明，通过HO-GSVD进行选择，合并后模型的性能优于随机选择 [cite: 182, 205]。

实验与结论

[cite_start]论文通过在多个视觉分类任务（最多20个任务）和不同规模的ViT模型上的大量实验，验证了其方法的有效性 [cite: 115, 136]。

- [cite_start]**Subspace Boosting**： 在所有测试的基线方法（Task Arithmetic, TIES-Merging, Consensus Merging）上都带来了持续且显著的性能提升，即使与正交的增强技术LiNeS结合使用，效果依然明显 [cite: 117, 150]。

- [cite_start]**HO-GSVD**：不仅为模型合并提供了可解释的视角，证明了浅层网络学习通用特征、深层网络学习特定任务特征的观点 [cite: 298][cite_start]，还成功地被用于优化专家模型的选择，以获得更好的合并性能 [cite: 209]。

结论：论文成功地将模型合并性能下降的问题归因于“秩崩溃”，并提出了简单有效的 Subspace Boosting 方法来缓解此问题。更进一步，它引入了HO-GSVD，为理解和优化模型合并过程提供了强大的可解释性工具。